
Building an LLM in C++

From First Principles to GPU-Accelerated Chatbot

A complete technical guide covering theory, mathematics, and implementation

Abstract

This document presents a complete derivation and implementation of a Generative Pre-trained Transformer (GPT) language model built entirely in C++, accelerated by NVIDIA CUDA. Beginning from fundamental concepts accessible to readers without a machine learning background, the guide progresses systematically through tokenization, embedding theory, the attention mechanism, backpropagation, and GPU-resident training. The final implementation targets an RTX 3050 Laptop GPU (6 GB VRAM, Ampere SM86) and scales to 10.7 million parameters, matching the nanoGPT Shakespeare configuration. Full C++ source code is provided and annotated throughout.

Field	Detail
Author	Introvert Programmer
Project Language	C++17 with CUDA
Target Hardware	NVIDIA RTX 3050 Laptop GPU, 6 GB VRAM, Ampere SM86
Model Scale	10.7 million parameters (nanoGPT Shakespeare configuration)
Training Speed	Approx. 2 to 5 ms per iteration (vs. 250 ms on CPU)
Architecture	GPT-style Transformer: Pre-Norm, Multi-Head Attention, GELU, Adam
Tokenizer	Character-level (65 tokens for Shakespeare dataset)
Source Files	matrix.hpp, model.hpp, tokenizer.hpp, optimizer.hpp, main.cpp, chat.cpp, cuda_kernels.cu / .cuh

Table of Contents

Part 1: What Is a Large Language Model?	4
1.1 The Core Idea	4
1.2 Why This Approach Works.....	4
1.3 The Three Phases of Building ChatGPT.....	4
1.4 What Building From Scratch Means	5
Part 2: Neural Networks and How They Learn.....	6
2.1 What Is a Neuron?	6
2.2 Layers and Depth.....	6
2.3 How Weights Are Learned	6
Part 3: Tokenization	8
3.1 Why Text Must Become Numbers	8
3.2 Character-Level Tokenization	8
3.3 Byte-Pair Encoding: What Production Models Use	8
Part 4: Embeddings and Positional Encoding	10
4.1 From Token ID to Vector	10
4.2 Positional Encoding.....	10
Part 5: The Attention Mechanism	11
5.1 The Problem Attention Solves.....	11
5.2 Query, Key, and Value	11
5.3 The Attention Formula, Step by Step.....	11
5.4 Multi-Head Attention.....	12
5.5 GPU Acceleration of Attention	12
Part 6: The Transformer Block.....	14
6.1 Layer Normalization	14
6.2 Residual Connections	14
6.3 The Feed-Forward Network	15
6.4 GELU Activation	15
Part 7: Loss Function and Backpropagation.....	16
7.1 The Output Layer	16
7.2 Cross-Entropy Loss.....	16
7.3 The Gradient of Softmax and Cross-Entropy Combined	16
7.4 Backpropagation	17
7.5 Gradient Clipping.....	17
Part 8: The Adam Optimizer and Learning Rate Scheduling	18
8.1 Plain Gradient Descent and Its Limitations	18

8.2 Adam: Adaptive Moment Estimation.....	18
8.3 GPU-Resident Optimizer.....	18
8.4 Learning Rate Scheduling.....	19
Part 9: GPU Architecture and CUDA Implementation.....	20
9.1 Why GPU Training.....	20
9.2 GPU-Backed Matrix Design.....	20
9.3 CUDA Kernel Overview.....	20
9.4 Gradient Accumulation.....	21
Part 10: Complete Model Architecture.....	22
10.1 Hyperparameters.....	22
10.2 Forward Pass Data Flow.....	22
10.3 File Responsibilities.....	23
Part 11: Training Loop.....	24
11.1 Step-by-Step Training.....	24
11.2 Training Hyperparameters and Expected Results.....	24
Part 12: Our Model Compared to GPT-2.....	25
12.1 Architecture Comparison.....	25
12.2 What Is Identical.....	25
12.3 What Scale Provides.....	25
12.4 Why Our Perplexity Is Lower.....	26
12.5 The Path from Our Model to GPT-2.....	26
Part 13: Build Instructions and Common Issues.....	27
13.1 Prerequisites.....	27
13.2 CMakeLists.txt.....	27
13.3 Build and Run.....	27
13.4 Common Issues and Fixes.....	27
Part 14: Learning Roadmap and Key References.....	29
14.1 Recommended Next Steps.....	29
14.2 Essential Reading.....	29

Part 1: What Is a Large Language Model?

1.1 The Core Idea

A large language model is a computer program that predicts the next token in a sequence. That is its entire function. Given all the tokens it has seen so far, it assigns a probability to every possible next token and selects one. It repeats this process token by token until it produces a complete response.

This sounds deceptively simple. The power comes from scale. After training on hundreds of billions of tokens drawn from books, websites, code repositories, and scientific articles, the probability distributions the model learns are so rich and interconnected that the output reads as fluent, knowledgeable, and contextually appropriate text.

The Autocomplete Analogy

Picture the autocomplete on a smartphone keyboard. It watches what you type and suggests the next word. A large language model does the same thing, except it has read virtually everything ever written and its suggestions span entire paragraphs of nuanced, domain-specific prose.

1.2 Why This Approach Works

Before 2017, chatbots operated on hand-crafted rules. A programmer would write: if the user says hello, reply with hi. These systems broke the moment a user said anything unexpected. They could not generalise beyond their explicitly programmed cases.

Language models learn from data instead of rules. Nobody tells the model what grammar is. Nobody tells it that Paris is the capital of France. It infers everything by noticing statistical patterns across billions of examples. The patterns it discovers are so numerous and fine-grained that they encode vocabulary, syntax, factual knowledge, reasoning style, and conversational conventions simultaneously.

Property	Rule-Based Chatbot	Language Model (Our Approach)
Learning mechanism	Manually programmed rules	Statistical patterns from training data
Handles unexpected input	Fails or falls to default response	Generalises to new phrasing naturally
Knowledge source	What the programmer wrote	Everything in the training corpus
Flexibility	Fixed and deterministic	Probabilistic and adaptable
Maintenance	Requires manual updates	Improves with more data

1.3 The Three Phases of Building ChatGPT

1. Pre-training. The model reads enormous quantities of text and learns to predict the next token. This is where language, knowledge, and reasoning ability are acquired. It is what our C++ implementation builds from scratch.
2. Fine-tuning. The pre-trained model is trained further on high-quality question-and-answer pairs so it behaves as a helpful assistant rather than a plain text predictor.
3. Reinforcement Learning from Human Feedback (RLHF). Human evaluators rate the model outputs. Those ratings train the model to be more helpful, accurate, and safe.

This guide covers Phase 1 in full. Every formula, every variable, and every line of C++ code in the pre-training pipeline is explained here. Understanding Phase 1 accounts for roughly ninety-five percent of the engineering complexity.

1.4 What Building From Scratch Means

Most practitioners use PyTorch or TensorFlow, which hide thousands of lines of C++ behind convenient function calls. When a PyTorch program calls `loss.backward()`, an entire automatic differentiation engine runs invisibly. Our project writes every one of those lines explicitly: every matrix multiplication, every gradient computation, every weight update. The GPU kernels themselves are authored in CUDA. Nothing is a black box.

Part 2: Neural Networks and How They Learn

2.1 What Is a Neuron?

A neuron is a small mathematical function with one job: take several numbers as input, multiply each by a personal weight, sum the results, add a bias offset, and pass the total through a nonlinear activation function. The output is a single number that flows into the next layer.

Think of a neuron that decides whether a film review is positive. It receives counts of words like excellent, boring, loved, and disappointing. Each word has a learned weight representing how strongly it implies positivity or negativity. The bias represents the neuron's default lean before reading any words. The activation function converts the weighted sum into a final signal.

```
output = activation( w1*x1 + w2*x2 + ... + wn*xn + bias )
```

Symbol	Name	Meaning
$x_1 \dots x_n$	Inputs	The numbers entering this neuron from the previous layer or from raw data
$w_1 \dots w_n$	Weights	How much importance this neuron assigns to each input. Learned during training.
bias	Bias	A constant offset. Allows the neuron to activate even when all inputs are zero.
activation	Activation function	A nonlinear function (GELU in our model) that decides the output scale.
output	Neuron output	The single number passed to the next layer.

2.2 Layers and Depth

Neurons are grouped into layers. The first layer receives token embeddings. Each subsequent layer takes the previous layer's output as input. The final layer produces the probability distribution over the next token. In between lie the hidden layers. Our model has six of these transformer layers stacked in sequence. GPT-4 has ninety-six.

Depth matters because each layer learns increasingly abstract features. Early layers detect low-level patterns such as common letter sequences. Later layers understand high-level concepts such as subject-verb agreement or thematic consistency across sentences.

2.3 How Weights Are Learned

At the start of training all weights are initialised to small random values. The model makes a prediction, and a loss function measures how wrong that prediction was. Backpropagation traces the error backward through every layer and computes how much each weight contributed to the mistake. The

Adam optimizer then nudges each weight by a small amount in the direction that reduces the loss. This cycle repeats millions of times.

The Central Insight

There is no clever shortcut to training a language model. The process is the same five-step loop repeated billions of times on billions of examples: predict, measure error, compute gradients, update weights, repeat. The intelligence that emerges is entirely a consequence of scale and the richness of the training data.

Part 3: Tokenization

3.1 Why Text Must Become Numbers

Neural networks operate on floating-point numbers. Raw text cannot enter a matrix multiplication directly. A tokenizer converts each unit of text into an integer identifier, and the model maps that integer to a dense vector of numbers. Everything the model reads, produces, and learns from is expressed as arrays of floating-point values.

3.2 Character-Level Tokenization

Our implementation uses the simplest possible tokenizer. Every unique character in the training text receives an integer identifier. For the complete works of Shakespeare the vocabulary contains sixty-five characters: uppercase and lowercase letters, digits, punctuation marks, spaces, and newline characters.

Input text	Tokenized form	Integer IDs
Hello	H e l l o	22 43 51 51 57
ROMEO:	R O M E O :	35 22 20 12 22 4
To be,	T o b e ,	44 53 1 40 43 3

The CharTokenizer struct in tokenizer.hpp stores two maps. The stoi map converts any character to its integer ID. The itos map reverses the process, converting integer IDs back to characters. Both maps are serialized into model.bin so the chat program can decode model output without access to the original training file.

3.3 Byte-Pair Encoding: What Production Models Use

GPT-2, GPT-3, GPT-4, and LLaMA all use Byte-Pair Encoding (BPE). BPE starts with individual characters and then repeatedly merges the most frequently co-occurring adjacent pair into a single new token. After fifty thousand or more such merges, the vocabulary contains common words, word fragments, and morphemes as single tokens.

The word programming would be tokenized as two tokens under BPE rather than eleven characters. A context window of the same length therefore holds roughly four times as much information. This is why production models with BPE tokenization can maintain coherence across much longer passages than our character-level model with an identical context window.

Property	Character-Level (this project)	BPE (GPT-2 and later)
Vocabulary size	65 tokens	50,257 tokens

Token for the word programming	11 tokens (one per character)	2 tokens (program + ming)
Context efficiency	1 token = 1 character	1 token = approx. 4 characters on average
Implementation size	Approx. 40 lines of C++	Approx. 800 to 1,500 lines of C++
Used in production	No	Yes: GPT-2, GPT-3, GPT-4, LLaMA

Why We Use Character-Level Tokenization Here

BPE requires a multi-pass corpus scan to build merge rules before training begins and significantly increases the embedding table size. Character-level tokenization is buildable in forty lines of C++ and makes the token representation immediately visible during debugging. Every concept explained in this guide applies identically to a BPE tokenizer. The architecture does not change.

Part 4: Embeddings and Positional Encoding

4.1 From Token ID to Vector

An integer token ID carries no structural information. The integer 42 bears no mathematical relationship to the integer 43 despite the characters they represent possibly being semantically related. An embedding replaces each integer with a dense vector of `d_model` floating-point values. These values are learned during training and gradually encode semantic and syntactic relationships.

After training, tokens that appear in similar contexts acquire similar embedding vectors. The vectors for the words king and queen will be close together in the `d_model`-dimensional space. This proximity emerges entirely from gradient descent and the patterns in the training data. No human assigns these positions.

```
embedding = wte[ token_id ]    -- row lookup in the token embedding table
```

Symbol	Shape	Meaning
wte	(vocab_size x d_model) = (65 x 384)	Token embedding table. Each row is the learned vector for one token.
token_id	Integer in range [0, vocab_size)	The integer assigned to a character by the tokenizer.
embedding	(1 x d_model) = (1 x 384)	The dense vector representation of this token. All learning starts here.
d_model	384 (our scaled model)	The width of every internal representation throughout the model.

4.2 Positional Encoding

The attention mechanism treats every position in a sequence identically unless told otherwise. Without positional information the sentence the dog bites the man would be indistinguishable from the man bites the dog. Our model uses learned positional embeddings. A second table called wpe contains one row per possible position, up to `block_size` positions. The embedding for a token at position `t` is the sum of its token embedding and its positional embedding.

```
input[t] = wte[ token_id[t] ] + wpe[ t ]
```

Both tables are initialized with small random values and learned jointly during training. The model learns to place information about word order into the positional rows of wpe and semantic content into the token rows of wte.

Part 5: The Attention Mechanism

5.1 The Problem Attention Solves

Consider the sentence: the animal did not cross the street because it was too tired. When processing the pronoun it, a reader instantly understands the referent is animal, not street. Understanding this requires connecting two words separated by several positions. Earlier neural architectures compressed the entire preceding context into a single fixed-size vector before processing each word, making long-range connections extremely difficult to learn.

Attention allows every token to examine every other token in the sequence simultaneously and decide, through learned weights, how much information to draw from each. The token it can look directly back at animal regardless of distance.

5.2 Query, Key, and Value

Each token produces three vectors from its embedding through learned linear projections: a Query, a Key, and a Value. These names come from the analogy of a search engine. The Query represents what this token is looking for. The Key represents what each token advertises about itself. The Value represents the information a token provides when retrieved.

```
Q = X * W_Q      shape: (T x d_model) * (d_model x d_k) = (T x d_k)
K = X * W_K      same shape as Q
V = X * W_V      same shape as Q
```

Symbol	Shape in our model	Meaning
X	(T x 384)	Input matrix. One row per token, containing the summed token and position embedding.
W_Q, W_K, W_V	(384 x 64)	Learned projection matrices. One set per attention head. Each head learns different projections.
Q, K, V	(T x 64)	Query, Key, and Value matrices for all T tokens in this head.
T	Up to 256	The number of tokens currently in the context window (block_size).
d_k	64 = 384 / 6	The dimension of each head. Total is d_model because d_k times n_heads equals d_model.

5.3 The Attention Formula, Step by Step

Step 1: Compute relevance scores

For every pair of tokens i and j, compute the dot product of their Query and Key vectors. A large dot product means the two vectors point in similar directions, indicating that token i should attend strongly

to token j . Divide by the square root of d_k to prevent the dot products from growing so large that the softmax saturates.

```
scores = Q * K^T * (1 / sqrt(d_k))    shape: (T x T)
```

Step 2: Apply the causal mask

During training the model must not use information from future positions when predicting the current token. Setting future positions to a very large negative value before applying softmax ensures those positions receive essentially zero weight.

```
scores[i][j] = -1e9    when j > i    (position j is in the future)
scores[i][j] = unchanged    when j <= i    (current or past position)
```

Step 3: Softmax to attention weights

Softmax converts each row of scores into a probability distribution. Every row sums to exactly one. Future positions, having received the value negative one billion, contribute probability approximately zero.

```
weights = softmax(scores)    shape: (T x T)
softmax(z_i) = exp(z_i) / sum( exp(z_j) for all j )
```

Step 4: Retrieve Values

The output for each token is the weighted average of all Value vectors, where the weights are the attention probabilities. Tokens judged highly relevant contribute more of their Value content to the output.

```
output = weights * V    shape: (T x T) * (T x d_k) = (T x d_k)
```

5.4 Multi-Head Attention

The entire attention procedure runs $h = 6$ times in parallel, each instance using its own independent W_Q , W_K , and W_V matrices. Each head specialises through training to track a different type of relationship: one head may connect pronouns to their antecedents, another may relate verbs to their subjects, a third may track positional proximity. The outputs of all heads are concatenated and projected back to d_{model} .

```
MultiHead(Q,K,V) = Concat(head_1, ..., head_6) * W_O
```

Why Multiple Heads?

Language contains many overlapping relationship types simultaneously. A single attention head can only learn one weighting scheme at a time. Six heads running in parallel let the model track six distinct relationship types simultaneously, producing a far richer representation than a single head could achieve with the same total compute budget.

5.5 GPU Acceleration of Attention

In the CUDA implementation the causal mask is applied by a dedicated kernel `cuda_causal_mask`, which writes negative one billion to all positions above the diagonal in parallel across the GPU. Matrix multiplications for the Q, K, V projections and the final value aggregation are dispatched to cuBLAS via `cuda_matmul`. The softmax kernel `cuda_softmax` uses shared memory reduction to compute the row maximum and normalization in a single GPU pass.

Part 6: The Transformer Block

6.1 Layer Normalization

As activations pass through many layers they tend to grow or shrink in magnitude. After enough layers the values can become enormous or collapse to near zero, destabilising training. Layer normalization rescales each token's vector to have zero mean and unit variance, then applies learned scale (gamma) and shift (beta) parameters.

```
mean    = (1 / d) * sum( x[j] for j = 0 .. d-1 )
var      = (1 / d) * sum( (x[j] - mean)^2 )
x_norm  = (x - mean) / sqrt(var + eps)          where eps = 1e-5
output  = gamma * x_norm + beta
```

Symbol	Value / Shape	Purpose
gamma	Vector of length d_model, init 1.0	Learned scale. Allows the model to recover a non-unit variance if the next layer needs it.
beta	Vector of length d_model, init 0.0	Learned shift. Allows the model to recover a non-zero mean.
eps	1e-5	Added to the variance before taking the square root. Prevents division by zero when variance is exactly zero.
inv_std_cache	Vector of T values, stored during forward	The backward pass divides by this value. Caching avoids recomputing the square root during backpropagation.

6.2 Residual Connections

Instead of replacing the input with the sub-layer output, a transformer block adds the sub-layer output back onto the original input. This is the residual connection. It creates a direct path for gradients to travel backward through the network without passing through any nonlinear transformations.

```
x1 = x + Attention( LayerNorm(x) )    first residual
x2 = x1 + FFN( LayerNorm(x1) )       second residual
```

Why Residuals Are Essential

In a six-layer network without residual connections, every gradient must pass through six nonlinear layers during backpropagation. Each layer shrinks the gradient slightly. By the time the gradient reaches the first layer it may be too small to cause any meaningful weight update. This is the vanishing gradient problem. Residual connections give gradients a shortcut that bypasses the nonlinearities, making deep networks trainable.

6.3 The Feed-Forward Network

After attention has mixed information between tokens, each token is processed independently by a two-layer feed-forward network. The first layer expands the representation from d_{model} to $d_{\text{ff}} = 4$ times d_{model} . The GELU activation introduces nonlinearity. The second layer compresses back to d_{model} .

$$\text{FFN}(\mathbf{x}) = \text{GELU}(\mathbf{x} * \mathbf{W1} + \mathbf{b1}) * \mathbf{W2} + \mathbf{b2}$$

Symbol	Shape in our model	Purpose
W1	(384 x 1536)	Expands each token from 384 to 1536 dimensions. This is where the model stores factual associations.
W2	(1536 x 384)	Compresses back to 384 so the residual connection can sum FFN output with the input.
b1, b2	Bias vectors	Allow each neuron to have a non-zero default output when all inputs are zero.
$d_{\text{ff}} = 1536$	$4 \times d_{\text{model}}$	The four-times expansion is a standard ratio. It provides sufficient capacity without excessive compute.

6.4 GELU Activation

GELU (Gaussian Error Linear Unit) is the activation function used inside the feed-forward network. Unlike ReLU, which hard-clips all negative values to zero, GELU applies a smooth gate that allows small negative values to pass through with reduced magnitude. This prevents the dead neuron problem and produces better gradient flow near zero.

$$\text{GELU}(\mathbf{x}) = 0.5 * \mathbf{x} * (1 + \tanh(0.7978845608 * (\mathbf{x} + 0.044715 * \mathbf{x}^3)))$$

The constant 0.7978845608 is the square root of two divided by pi. The polynomial approximation inside the tanh closely matches the exact GELU formulation (which requires evaluating the Gaussian cumulative distribution function) while executing far more efficiently.

Part 7: Loss Function and Backpropagation

7.1 The Output Layer

After all six transformer blocks have processed the sequence, a final LayerNorm stabilises the representation. A linear projection called `lm_head` then maps from `d_model` dimensions to `vocab_size` dimensions, producing one raw score per token in the vocabulary for every position.

```
logits = final_hidden * lm_head      shape: (T x d_model) * (d_model x 65) = (T x 65)
```

These raw scores are called logits. Softmax converts each row into a probability distribution over the sixty-five possible next characters. During generation the model samples from this distribution (with temperature scaling). During training it compares the distribution to the known correct next character.

7.2 Cross-Entropy Loss

The cross-entropy loss measures how surprised the model was by the correct answer. If the model assigned probability 0.9 to the correct character the loss is low. If it assigned probability 0.01 the loss is high.

```
loss = -(1 / T) * sum( log( P(y[t]) ) for t = 0..T-1 )
```

Symbol	Meaning
T	The number of tokens in the current context window (<code>block_size = 256</code>)
<code>y[t]</code>	The index of the correct next character at position <code>t</code>
<code>P(y[t])</code>	The probability the model assigned to that correct character after softmax
log	Natural logarithm. Converts small probabilities into manageable negative numbers.

7.3 The Gradient of Softmax and Cross-Entropy Combined

Computing the gradient of the loss with respect to the logits requires applying the chain rule through both the softmax and the log operations. When derived together, most terms cancel and the result is elegant.

```
d_logits[t][j] = probs[t][j]      for all j
d_logits[t][ y[t] ] -= 1.0          subtract one at the correct position
d_logits[t][j] /= block_size        normalise by sequence length
```


For the correct token the gradient is the predicted probability minus one. If the model is very confident and correct, the gradient is small. If the model is very wrong, the gradient is large and negative, pushing the weight update to increase that token's probability. For all incorrect tokens the gradient equals the predicted probability, pushing their probabilities downward.

7.4 Backpropagation

Backpropagation applies the chain rule of calculus in reverse through every operation in the network. Each module's backward method receives the gradient of the loss with respect to its output and returns the gradient with respect to its input, while accumulating the gradients for its own parameters.

```

d_V      = weights^T * d_out      gradient through value aggregation
d_scores = softmax_backward(d_out * V^T)  gradient through softmax
d_Q      = d_scores * K          gradient for queries
d_K      = d_scores^T * Q        gradient for keys
dW_Q     = X^T * d_Q             weight gradient for query projection
                                dW_K = X^T * d_K
                                dW_V = X^T * d_V
d_X      = d_Q * W_Q^T + d_K * W_K^T + d_V * W_V^T  input gradient

```

7.5 Gradient Clipping

Occasionally a training example produces unusually large gradients. If applied directly these could destabilise the model by pushing weights far from their current values. Gradient clipping scales all gradients proportionally so their combined magnitude does not exceed a fixed threshold.

```

global_norm = sqrt( sum( g^2 for every gradient g ) )
if global_norm > 1.0:  scale = 1.0 / global_norm,  g = g * scale  for all g

```

The direction of the gradient update is preserved. Only the magnitude is reduced. This makes training robust to the occasional problematic batch without affecting normal training steps.

Part 8: The Adam Optimizer and Learning Rate Scheduling

8.1 Plain Gradient Descent and Its Limitations

The simplest update rule subtracts the gradient directly from each weight: $W = W - \text{lr} \times \text{gradient}$. This treats every weight identically. A weight that has consistently small gradients receives the same sized step as a weight with large erratic gradients, which is rarely optimal.

8.2 Adam: Adaptive Moment Estimation

Adam maintains two exponential moving averages for every weight. The first moment m tracks the mean of recent gradients, providing momentum. The second moment v tracks the mean of recent squared gradients, providing an estimate of each weight's gradient volatility.

$m[i] = \text{beta1} * m[i] + (1 - \text{beta1}) * \text{grad}[i]$	update mean
$v[i] = \text{beta2} * v[i] + (1 - \text{beta2}) * \text{grad}[i]^2$	update variance
$\text{lr_t} = \text{lr} * \text{sqrt}(1 - \text{beta2}^t) / (1 - \text{beta1}^t)$	bias-corrected rate
$\text{params}[i] -= \text{lr_t} * m[i] / (\text{sqrt}(v[i]) + \text{eps})$	weight update

Symbol	Value	Purpose
lr	6e-4 (GPU training)	Base learning rate. Higher than typical CPU training because gradient accumulation over 32 micro-batches stabilises updates.
beta1	0.9	Decay rate for the gradient mean. Weights 90% history, 10% current gradient. Smooths noisy estimates.
beta2	0.999	Decay rate for the gradient variance. Changes slowly, accumulating a long history of gradient magnitudes.
eps	1e-8	Prevents division by zero when a weight has received near-zero gradients and v is approximately zero.
t	Timestep counter	Used in bias correction. Early in training m and v underestimate their true values because they start at zero. Dividing by $(1 - \text{beta}^t)$ corrects this.

8.3 GPU-Resident Optimizer

In the CUDA implementation the moment vectors m and v live entirely in GPU memory as raw float pointers d_m and d_v . The step function calls the `cuda_adam_update` kernel, which executes the complete Adam update for all parameters in a single GPU dispatch. No data travels across the PCIe bus during the optimizer step, eliminating a significant bottleneck present in CPU-side optimizers.

8.4 Learning Rate Scheduling

The learning rate varies over the course of training. A warmup phase linearly increases it from zero to the peak value over the first two hundred steps. After warmup a cosine decay schedule smoothly reduces it to the minimum value by the end of training.

```
Warmup (step < 200): lr = max_lr * (step + 1) / warmup_steps
Cosine decay: decay_ratio = (step - warmup) / (max_steps - warmup)
               coeff = 0.5 * (1 + cos(pi * decay_ratio))
               lr = min_lr + coeff * (max_lr - min_lr)
```

The warmup prevents instability at the start of training when weights are random and Adam moment estimates are zero. The cosine decay provides large steps early when the model is far from a good solution and progressively smaller steps as it converges.

Part 9: GPU Architecture and CUDA Implementation

9.1 Why GPU Training

Training on CPU with the scaled 10.7 million parameter model takes approximately 245 milliseconds per iteration. Running 5,000 optimizer steps with 32 gradient accumulation micro-batches would require over eight hours. The same training on an RTX 3050 Laptop GPU takes approximately 2 to 5 milliseconds per iteration, reducing total training time to under ten minutes.

GPUs achieve this speedup because matrix multiplication, the dominant operation in transformer training, is embarrassingly parallel. A 384 by 1536 matrix multiply requires roughly 590,000 independent multiply-accumulate operations. The RTX 3050 Laptop GPU contains 2,048 CUDA cores that execute these in parallel.

9.2 GPU-Backed Matrix Design

The Matrix struct in matrix.hpp was redesigned for the GPU version. Instead of a CPU `std::vector` the primary storage is a raw GPU pointer `d_data` allocated via `cuda_alloc`. The CPU vector is used only for transfers. All arithmetic operations call CUDA kernels directly on `d_data`.

```
struct Matrix {
    int rows = 0, cols = 0;
    float* d_data = nullptr; // GPU memory (primary storage)

    Matrix(int r, int c) : rows(r), cols(c) {
        d_data = cuda_alloc(r * c); // allocate on GPU
    }
    Matrix matmul(const Matrix& B) const {
        Matrix C(rows, B.cols);
        cuda_matmul(d_data, B.d_data, C.d_data, rows, cols, B.cols);
        return C; // result stays on GPU
    }
};
```

9.3 CUDA Kernel Overview

Kernel	Operation	Acceleration Method
cuda_matmul	Matrix multiplication $C = A * B$	Dispatches to cuBLAS cublasSgemm. Uses Tensor Cores on Ampere for FP16 mixed precision.
cuda_softmax	Row-wise normalization softmax	Shared memory reduction finds row maximum and sum in a single kernel pass without intermediate global memory writes.
cuda_causal_mask	Set future positions to negative infinity	One thread per matrix element above the diagonal. Fully parallel with no dependencies.

cuda_adam_update	Adam optimizer step for all parameters	One thread per parameter. Reads m, v, grad; writes new m, v, param in registers. Zero PCIe traffic.
cuda_add, cuda_scale	Element-wise add and scalar multiply	One thread per element. Used for gradient accumulation and residual connections.
cuda_layernorm	Fused layer normalization	Computes mean and variance in shared memory, normalizes and applies gamma and beta in one pass.

9.4 Gradient Accumulation

The GPU version uses gradient accumulation to simulate a large effective batch size without requiring all samples to fit in VRAM simultaneously. Each optimizer step processes 32 micro-batches. Gradients from each micro-batch are summed on the GPU using `cuda_add`. After all 32 micro-batches the accumulated gradients are divided by 32 and passed to the Adam optimizer.

```
effective_batch_size = 32 micro-batches * 256 tokens = 8,192 tokens per step
```

This is equivalent to training with a batch size of 8,192 tokens while only ever holding one 256-token sequence in VRAM at a time. The larger effective batch size stabilises training and allows a higher peak learning rate.

Part 10: Complete Model Architecture

10.1 Hyperparameters

The model is configured to match the nanoGPT Shakespeare configuration. All six files share these values through the constants defined at the top of main.cpp.

Parameter	Symbol	Our GPU Model	GPT-2 Small
Vocabulary size	vocab_size	65 (character-level)	50,257 (BPE)
Embedding dimension	d_model	384	768
Transformer layers	n_layers	6	12
Attention heads	n_heads	6	12
Head dimension	d_k	64 (= 384 / 6)	64
Feed-forward width	d_ff	1,536 (= 4 x 384)	3,072
Context window	block_size	256 tokens	1,024 tokens
Total parameters		Approx. 10.7 million	117 million
Gradient accumulation	accum_steps	32 micro-batches	N/A
Peak learning rate	max_lr	6e-4	Varies

10.2 Forward Pass Data Flow

```
Input tokens: [44, 53, 1, 40, 43, ...]  integers in range [0, 64]
```

```
1. Token + positional embeddings
   x[t] = wte[ token_id[t] ] + wpe[ t ]
   shape: (T x 384)    all on GPU

2. Six Transformer Blocks (sequentially)
   ln1_out = LayerNorm1( x )
   attn_out = MultiHeadAttention( ln1_out )
   x        = x + attn_out
   ln2_out = LayerNorm2( x )
   ffn_out = FeedForward( ln2_out )
   x        = x + ffn_out

3. Final LayerNorm
   x = ln_f.forward( x )

4. Language model head
   logits = x * lm_head  shape: (T x 65)

5. Softmax + sample
   probs = softmax( logits / temperature )
```

```
next_token = sample from probs[T-1]
```

10.3 File Responsibilities

File	Responsibility
matrix.hpp	Matrix struct with GPU pointer d_data. All arithmetic routes to CUDA kernels.
cuda_kernels.cuh	C++ header declaring all CUDA kernel wrapper functions callable from .cpp files.
cuda_kernels.cu	CUDA implementation: cuda_matmul (cuBLAS), cuda_softmax, cuda_causal_mask, cuda_adam_update, and element-wise operations.
tokenizer.hpp	CharTokenizer with build, encode, decode, save, and load methods.
model.hpp	LayerNorm, AttentionHead, MultiHeadAttention, FeedForward, TransformerBlock, GPT. All operations GPU-resident.
optimizer.hpp	AdamOptimizer with GPU-resident moment vectors d_m and d_v. Single kernel dispatch per step.
main.cpp	Data loading, tokenizer, model and optimizer construction, gradient accumulation training loop, generation.
chat.cpp	Interactive terminal. Loads model.bin, encodes prompts, calls generate, decodes output.

Part 11: Training Loop

11.1 Step-by-Step Training

4. Update learning rate according to the cosine schedule.
5. Sample a random starting position in the encoded training data.
6. For each of the 32 gradient accumulation micro-batches: extract a 256-token input sequence `x` and its shifted target sequence `y`; call `model.zero_grad()`; run the forward pass to get logits; transfer logits to CPU; compute softmax and cross-entropy loss; compute `d_logits`; transfer `d_logits` to GPU; run the backward pass; gather micro-batch gradients via `gather_grads_gpu`; accumulate with `cuda_add` into `accum_grads`.
7. Divide `accum_grads` by 32 using `cuda_scale`.
8. Clip the gradient norm with `clip_grad_norm_gpu`.
9. Gather all parameters via `gather_params_gpu`.
10. Call `optimizer.step` with the GPU parameter and gradient pointers.
11. Write updated parameters back with `scatter_params_gpu`.

11.2 Training Hyperparameters and Expected Results

Iteration	Approx. Loss	Perplexity	Sample output character
0 (start)	4.17	65	Random characters with no pattern
500	2.9	18	Letter clusters beginning to form
1,000	2.5	12	Some recognisable words appear
2,500	2.1	8	Shakespearean style becoming visible
5,000	1.7 to 1.9	5 to 7	Coherent phrases with character names

Expected Training Speed on RTX 3050

With the 10.7 million parameter model and 32 gradient accumulation micro-batches, each optimizer step runs in approximately 100 to 200 milliseconds on the RTX 3050 Laptop GPU. Five thousand steps complete in roughly 10 to 15 minutes. This compares to approximately 27 hours for the same configuration on CPU.

Part 12: Our Model Compared to GPT-2

12.1 Architecture Comparison

Property	Our GPU Model	GPT-2 Small	GPT-2 Large
Total parameters	Approx. 10.7 million	117 million	774 million
Embedding dimension	384	768	1,280
Transformer layers	6	12	36
Attention heads	6	12	20
Feed-forward width	1,536	3,072	5,120
Context window	256 tokens	1,024 tokens	1,024 tokens
Tokenizer	Character-level (65 tokens)	BPE (50,257 tokens)	BPE (50,257 tokens)
Training data	1 MB Shakespeare	40 GB WebText	40 GB WebText
Training hardware	RTX 3050 Laptop GPU	Multiple A100 GPUs	Multiple A100 GPUs
Training time (approx.)	10 to 15 minutes	Weeks	Months

12.2 What Is Identical

Despite the difference in scale, our model and GPT-2 share the same architectural decisions at every level. Both use Pre-Norm placement of LayerNorm (applied before each sub-layer rather than after). Both use multi-head causal self-attention with the same scaled dot-product formula. Both use GELU activation in the feed-forward network. Both use cross-entropy loss on next-token prediction. Both use Adam with the same beta values. Both use gradient clipping by global L2 norm.

Understanding the code in this project means understanding GPT-2 completely. The barrier between this implementation and GPT-4 is scale and fine-tuning, not a different algorithm.

12.3 What Scale Provides

- More parameters mean more memory capacity. GPT-2 Small stores enough information to reproduce the style and vocabulary of thousands of authors across dozens of domains. Our model stores enough to approximate one author's style on one dataset.
- A longer context window means better coherence. GPT-2 can reference text written four times as many characters ago. Our model loses context within a few sentences.
- BPE tokenization means more information per context position. GPT-2 can fit roughly four times as much semantic content into the same number of context tokens.
- Larger training data means broader generalisation. Our model can only generate Shakespearean English and cannot encode any character not present in the training file.

12.4 Why Our Perplexity Is Lower

After 5,000 training iterations our model reaches perplexity around 5 to 7. GPT-2 Small reports perplexity around 35 on general English text. This does not mean our model is better. Perplexity is only comparable between models evaluated on the same dataset with the same vocabulary. Our model predicts from only 65 possible characters on a single author's text. GPT-2 predicts from 50,257 tokens across all domains of human writing. A model that has memorised one book will appear to outperform a model that understands all of human language when both are tested only on that book.

12.5 The Path from Our Model to GPT-2

Step	Change Required	Expected Impact
1	Replace Shakespeare (1 MB) with a diverse corpus (40+ GB)	Largest single improvement. Broad generalisation requires broad data.
2	Implement BPE tokenization with a 50,257-token vocabulary	Four times more semantic content per context position.
3	Scale to 12 layers, d_model = 768, 12 heads, d_ff = 3,072 (117 M params)	Model capacity sufficient to store broad factual knowledge.
4	Extend context window to 1,024 tokens	Multi-paragraph coherence and long-range reference tracking.
5	Add dropout regularization during training	Prevents overfitting on larger, more diverse datasets.
6	Multi-GPU data-parallel training	Necessary to complete training in weeks rather than years.

Part 13: Build Instructions and Common Issues

13.1 Prerequisites

- CUDA Toolkit 12.0 or later. Download from developer.nvidia.com/cuda-downloads. Select Windows, x86_64, 11, exe (local). Restart the terminal after installation and verify with `nvcc -version`.
- CMake 3.18 or later with CUDA language support.
- A C++17-compatible compiler: MSVC on Windows, GCC 9 or later on Linux.
- cuBLAS, which is included with the CUDA Toolkit installation.

13.2 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.18)
project(llm_cpp LANGUAGES CXX CUDA)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CUDA_ARCHITECTURES 86)    # RTX 3050 Laptop is SM86 (Ampere)

add_executable(llm src/main.cpp src/cuda_kernels.cu)
target_include_directories(llm PRIVATE src)
target_link_libraries(llm cublas)

add_executable(chat src/chat.cpp src/cuda_kernels.cu)
target_include_directories(chat PRIVATE src)
target_link_libraries(chat cublas)
```

13.3 Build and Run

```
# Download Shakespeare training data
mkdir data
curl -o data/input.txt https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt

# Build (Release mode is required for acceptable training speed)
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel 4

# Train the model (approx. 10 to 15 minutes on RTX 3050)
.\build\llm.exe

# Chat with the trained model
.\build\chat.exe model.bin
```

13.4 Common Issues and Fixes

Symptom	Likely Cause	Fix
---------	--------------	-----

Loss immediately becomes NaN	Learning rate too high or numerical overflow in a kernel	Reduce max_lr from 6e-4 to 3e-4 and rebuild.
Loss is constant after 500 steps	zero_grad not called, so gradients accumulate indefinitely	Verify that model.zero_grad() is called at the start of every micro-batch.
CUDA out of memory error	Model or batch too large for 6 GB VRAM	Reduce block_size from 256 to 128 or reduce d_model from 384 to 256.
nvcc not found during cmake	CUDA Toolkit not installed or not on PATH	Install CUDA Toolkit, restart terminal, verify nvcc --version returns a version number.
Generated text is a single repeated character	Temperature is set too low	Use temperature 0.8 in the chat program rather than values below 0.3.
Garbled characters in the Windows terminal	PowerShell default code page is not UTF-8	Run chcp 65001 before launching the executable, or add SetConsoleOutputCP(65001) to chat.cpp.
Very slow training, similar to CPU speed	Debug build selected instead of Release	Pass -DCMAKE_BUILD_TYPE=Release to cmake configuration command.

Part 14: Learning Roadmap and Key References

14.1 Recommended Next Steps

Stage	Topic	Key Concepts Covered
Completed	Character-level GPT in C++ with CUDA	Full Transformer, backpropagation, Adam, GPU kernels
Next: Stage 2	Byte-Pair tokenizer Encoding	Subword tokenization, merge rule tables, vocabulary scaling
Stage 3	Scale to 50 million parameters	VRAM management, mixed-precision FP16 training, cuBLAS tuning
Stage 4	Multi-GPU data-parallel training	NCCL all-reduce, gradient synchronization across devices
Stage 5	Instruction fine-tuning	Supervised fine-tuning on question-and-answer pairs
Stage 6	RLHF foundations	Reward modeling, proximal policy optimization

14.2 Essential Reading

- Attention Is All You Need. Vaswani et al., 2017. The original Transformer paper. Every architecture decision in this guide traces back to this work.
- Language Models are Unsupervised Multitask Learners. OpenAI, 2019. The GPT-2 paper. Describes the Pre-Norm variant and scaling to 1.5 billion parameters.
- Adam: A Method for Stochastic Optimization. Kingma and Ba, 2014. Derives the Adam optimizer and its bias correction from first principles.
- Layer Normalization. Ba et al., 2016. Introduces LayerNorm and demonstrates its advantages over batch normalization for sequence models.
- Training Compute-Optimal Large Language Models. Hoffmann et al., 2022. The Chinchilla paper. Establishes the data-to-parameter scaling law.
- nanoGPT. Andrej Karpathy, GitHub. The reference implementation that informed our model configuration.

End of Document

Introvert Programmer